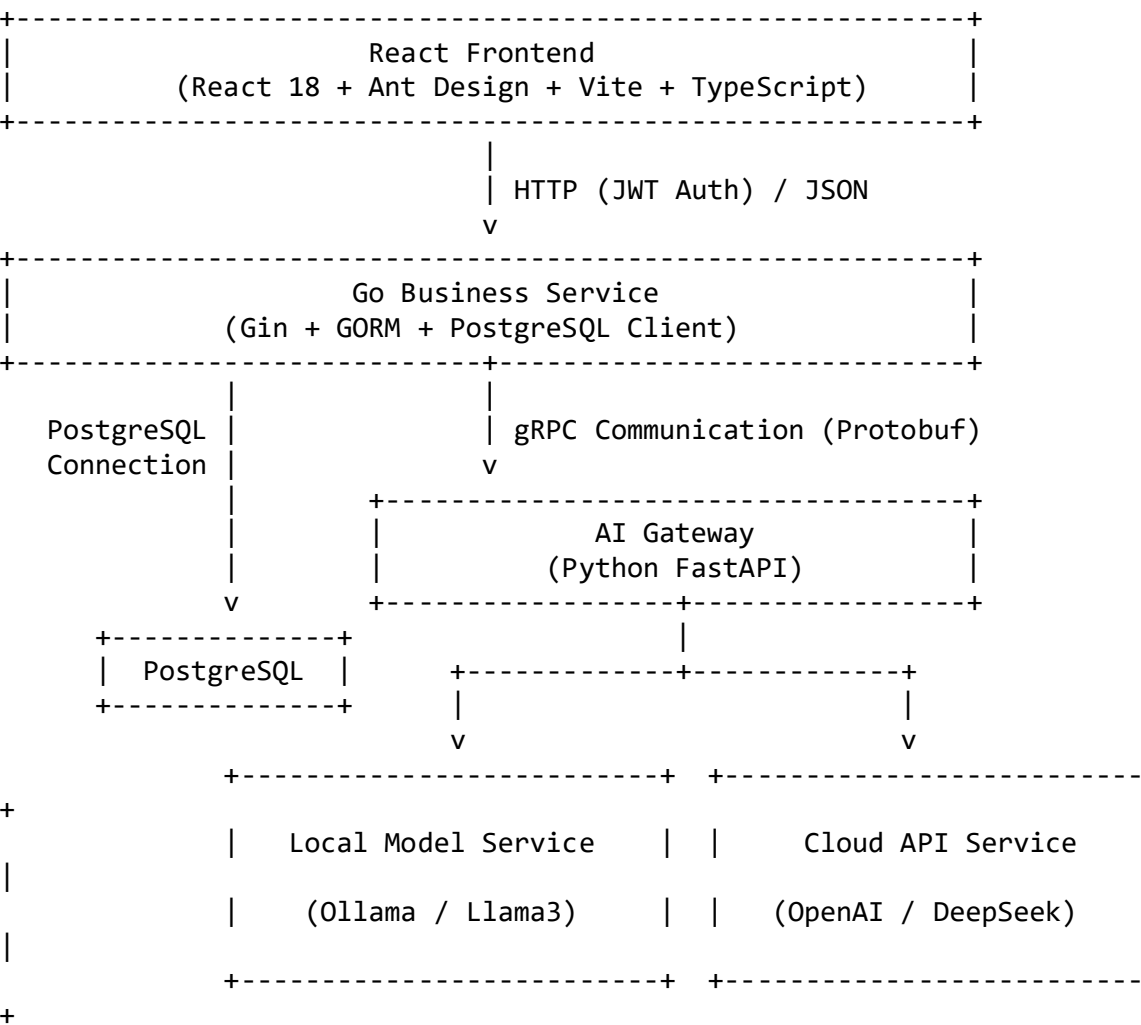


14 天 混合推理 AI 全栈项目开发计划书

本项目旨在开发一款支持混合推理模式（本地 Ollama 与云端 OpenAI/DeepSeek API 自动切换）的智能待办事项系统 (SmartTodo AI)。以下为针对《企业综合实训》制定的 14 天开发计划。

1. 项目架构设计 (Architecture)

1.1 架构拓扑图



1.2 AI 网关与配置热切换设计

为了在不重启 Python FastAPI 服务的情况下实现“本地模型 (Ollama)”与“云端大模型 (OpenAI/DeepSeek API)”之间的无缝热切换，系统采用以下设计方案：

- 统一的客户端抽象接口 (LLMInterface):** 定义统一的接口方法，如 `generate_chat_completion`，本地客户端与云端客户端均实现该接口。
- 内存配置管理器 (ConfigManager):** 使用 Python 字典在内存中维护当前的配置状态（包含 `active_provider`、`openai_api_key`、`openai_base_url`、`ollama_host` 等）。
- 配置文件监听 (Watchdog):** 利用 `watchdog` 库监控外部配置文件（如 `config.json` 或 `config.yaml`）的变化。当检测到文件被修改时，触发回调函数重新读取配置并更新内存中的 `ConfigManager`。
- 请求级动态分发 (Dynamic Routing):** 在 FastAPI 的路由处理函数中，不持有静态的客户端引用，而是通过依赖注入 (FastAPI Dependency Injection) 动态获取当前激活的客户端实例。这样每次请求都会根据最新的配置路由到对应的提供商，从而实现热切换。

2. 环境准备清单 (Day 0)

工具/服务	推荐版本	用途说明
Docker Desktop	20.10.x+	用于本地容器化编排数据库及前后端服务
Node.js & pnpm	18.x+ / 8.x+	前端 React 项目构建与依赖管理
Go SDK	1.20+	后端 Go 业务服务开发
Python & uv	3.10+	AI 网关 (FastAPI) 依赖与虚拟环境管理
Ollama	Latest	本地运行轻量级开源大模型 (Llama 3 / Qwen 2)
PostgreSQL	15.x	系统关系型数据库，存储用户和任务数据
Protobuf Compiler	3.x+	用于编译生成 Go 和 Python 的 gRPC 代码

3. 每日详细排期 (Core Timeline)

I. 基建与骨架阶段 (Day 1 - Day 3)

Day 1: 数据库与开发环境容器化部署

- 当日目标:** 完成本地开发环境的 Docker 基础架构搭建，确保 PostgreSQL 能够正常持久化运行。
- 具体任务:**
 - 编写项目的初始 `docker-compose.yml`，定义 PostgreSQL 服务，配置卷挂载实现数据持久化。
 - 创建初始化 SQL 脚本，预先配置好 `users` 和 `todos` 表的结构。
 - 启动容器，并使用客户端工具连接测试。

- **验收标准：**通过本地数据库管理工具成功连接 Docker 中的 PostgreSQL，并能够执行基本的建表与查询操作。

Day 2: Protobuf 接口定义与 gRPC 连通测试

- **当日目标：**定义 Go 与 Python 服务之间的通信契约，并验证跨语言 gRPC 通信。
- **具体任务：**
 - 编写 schema.proto，定义 AI 推理服务接口，包括输入任务描述、返回拆解子任务列表及时间预估的数据结构。
 - 编写 Makefile 脚本，调用 protoc 分别生成 Go 和 Python 的 gRPC 桩代码。
 - 编写最简 Go gRPC 客户端与 Python gRPC 服务端，实现一次 Hello World 级别的 RPC 调用。
- **验收标准：**运行 Go 客户端程序，能够在控制台正确打印输出 Python 服务端返回的响应数据。

Day 3: React 前端初始化与基础布局搭建

- **当日目标：**搭建前端单页应用骨架，配置路由与全局外观风格。
- **具体任务：**
 - 使用 Vite 初始化 React + TypeScript 项目，引入 Ant Design 组件库。
 - 配置 React Router，划分登录页、任务看板页以及 AI 配置面板页。
 - 使用 Ant Design 的 Layout 组件搭建后台管理系统的经典左侧导航与右侧内容区布局。
- **验收标准：**在浏览器中访问 <http://localhost:5173>，能够进行路由切换，并且侧边栏布局展示正常、无报错。

II. 核心业务阶段 (Day 4 - Day 8)

Day 4: 用户认证与 JWT 鉴权后端实现

- **当日目标：**完成基于 JWT 的无状态认证后端接口，保障系统安全性。
- **具体任务：**
 - 在 Go 业务端设计用户实体模型，使用 GORM 映射到 PostgreSQL。
 - 实现用户注册与密码加密存储 (bcrypt)。
 - 实现用户登录接口，校验成功后生成带有过期时间的 JWT Token，并编写 Gin 的 JWT 鉴权中间件。
- **验收标准：**使用 API 测试工具访问需要鉴权的接口，携带有效 Token 返回 200，无 Token 或 Token 过期返回 401。

Day 5: 任务 CRUD 后端接口设计与开发

- **当日目标：**完成待办事项的基础增删改查业务逻辑。
- **具体任务：**
 - 在 Go 项目中建立 Todo 的 GORM 模型，包含标题、描述、优先级、是否完成、预计耗时等字段。
 - 编写任务创建 POST /api/todos、删除 DELETE /api/todos/:id、修改 PUT /api/todos/:id 和列表获取 GET /api/todos 的 Gin 路由与控制器函数。
 - 编写单元测试验证 CRUD 逻辑。
- **验收标准：**CRUD 接口测试全部通过，数据库中能正确记录任务状态的变化。

Day 6: 前端用户中心与任务管理页面开发

- **当日目标：**实现用户登录注册前端交互及任务管理的主体界面。
- **具体任务：**
 - 开发登录与注册页面，使用 Ant Design Form 组件，并实现表单校验。
 - 在任务看板页使用 Ant Design Table 或 List 组件展示待办事项列表。
 - 开发“创建任务”与“编辑任务”的弹窗组件。
- **验收标准：**用户能够在前端完成登录，登录成功后跳转到任务列表页，能够正常看到列表框架。

Day 7: 前后端数据交互与接口联调

- **当日目标：**打通前端与 Go 后端的数据流，实现完整的业务闭环。
- **具体任务：**
 - 封装 Axios 请求模块，配置全局请求拦截器，在 Header 中自动携带 JWT Token。
 - 将登录、注册、任务列表、任务增删改查的 API 与 React 页面进行对接。
 - 处理前端的状态管理，在增删改操作后实时刷新任务列表。
- **验收标准：**在前端页面上操作添加、修改或删除任务，页面数据能够实时更新，且刷新页面后数据不丢失。

Day 8: 个人资料修改与密码重置功能闭环

- **当日目标：**完成用户个人资料的管理功能，结束核心业务阶段。
- **具体任务：**
 - 后端编写更新个人信息与修改密码的接口。
 - 前端开发个人资料修改页面，支持修改昵称、邮箱等，并提供旧密码校验以重置新密码。
 - 进行核心业务阶段的集成测试，修复已知的交互 Bug。

- **验收标准：**成功在前端修改用户密码，退出重新登录时必须使用新密码才能登录成功。
-

III. AI 双模集成阶段 (Day 9 - Day 12)

Day 9: Python FastAPI AI 网关搭建与双模切换设计

- **当日目标：**搭建 AI 微服务网关，实现双模驱动的底层路由逻辑。
- **具体任务：**
 - 使用 FastAPI 构建 Web 服务，同时后台运行 gRPC 服务（利用 Python 的多线程或 asyncio 任务）。
 - 实现本地模型（通过 LangChain/Ollama SDK 调用宿主机 Ollama）和云端大模型（通过 OpenAI/DeepSeek SDK 调用 API）的适配器。
 - 编写配置热加载机制，当接收到更新配置的请求时，更新内存中的模型调用指向。
- **验收标准：**通过 gRPC 调用 Python 网关，修改配置为本地模式时走 Ollama，修改为云端模式时成功调通外部 API。

Day 10: AI 智能任务拆解与时间预估 (JSON Mode 提示词工程)

- **当日目标：**实现核心 AI 业务，保证大模型返回结构化的任务拆解与耗时评估数据。
- **具体任务：**
 - 设计 System Prompt 和 User Prompt，采用 Few-Shot 方式引导模型输出指定结构的 JSON。
 - 开启模型的 JSON Mode，并在 Python 服务端使用 Pydantic 进行严格的数据 schema 校验，防止数据格式异常导致系统崩溃。
 - 在 Go 后端调用 gRPC 时，解析返回的 JSON，自动向 PostgreSQL 中插入拆解出来的子任务，并更新主任务的预测耗时。
- **验收标准：**在后台输入一个复杂任务，AI 能够成功返回包含“任务列表、时间预估、优先级”的结构化数据，且 Go 端能够顺利解析并落库。

Day 11: 悬浮球 ChatBot 全局助手与 RAG 知识库设计

- **当日目标：**实现前端悬浮球问答交互以及基于本地文档的轻量级 RAG。
- **具体任务：**
 - 前端开发右下角悬浮球组件，点击展开 Chat 窗口，支持流式 (SSE) 或普通 HTTP 问答。
 - 后端 Python 侧实现一个内存/文件级向量检索：读取用户上传的文本笔记，进行分段，利用轻量级 Embedding 模型（或简单的关键词检索）定位相关段落。
 - 拼接上下文，让本地模型回答用户关于笔记内容的提问。

- **验收标准：**上传一篇包含特定事实的文本文件，向悬浮球助手提问该事实，助手能基于上传的文档内容给出准确回答。

Day 12: 前端 AI 配置面板开发与整体 AI 功能联调

- **当日目标：**完成前端 AI 配置的可视化操作，实现全链路 AI 功能联调。
 - **具体任务：**
 - 前端开发 AI 配置面板，支持用户输入 API Key、Base URL，以及选择当前推理模式（云端、本地、自动降级）。
 - 配置发生变化时，通过 API 实时同步给 Python AI Gateway。
 - 调试在网络断开或 API 超时情况下，系统能否自动降级到本地 Ollama 模型进行任务拆解。
 - **验收标准：**在配置面板中切换模型源，后续的任务拆解和 Chat 服务会即时切换到对应的推理模型上，无需重启任何容器。
-

IV. 容器化与答辩阶段 (Day 13 - Day 14)

Day 13: 多阶段构建 Dockerfile 编写与 Docker Compose 统一编排

- **当日目标：**实现系统的一键容器化部署。
- **具体任务：**
 - 为 React 前端编写多阶段构建 Dockerfile（使用 Node 编译，Nginx 托管静态资源）。
 - 为 Go 业务端编写多阶段构建 Dockerfile（使用 golang 镜像编译出可执行二进制，再用 alpine/scratch 镜像运行）。
 - 为 Python AI 网关编写 Dockerfile（使用 uv/pip 安装依赖）。
 - 编写统一的 docker-compose.yml，配置自定义 bridge 网络，将前后端、AI 服务、数据库联结起来。
- **验收标准：**在项目根目录下执行 docker-compose up --build -d，所有容器均能正常启动，且各服务之间网络互通。

Day 14: 系统集成测试、Bug 修复与答辩汇报 PPT 梳理

- **当日目标：**完成系统交付，准备实训答辩。
- **具体任务：**
 - 进行全功能的黑盒测试，重点检查边界情况（如网络超时、非法 Token、大模型响应超时等）。
 - 修复测试过程中发现的缺陷。
 - 梳理项目技术亮点，制作包含架构图、双模切换原理、开发计划执行情况 & 运行效果截图的答辩 PPT。
- **验收标准：**系统稳定运行，无致命 Bug，答辩材料准备完毕。